



Pichulman Miguel Angel

TRABAJO PRÁCTICO 5: Optimización y Procesamiento de Datos

ACTIVIDAD 1: Optimización mediante Índices

1) Una empresa de logística gestiona miles de envíos. Las consultas más frecuentes buscan paquetes por su `numero_seguimiento` (único) y otras filtran por estado del envío y `ciudad_destino` simultáneamente.

- a) Identificar qué campo requiere un Índice Simple y proponer el comando para crearlo.

```
db.envios.createIndex({ numero_seguimiento: 1 })
```

- b) Identificar qué campos requieren un Índice Compuesto para optimizar las búsquedas por estado y ciudad. Proponer el comando.

```
db.envios.createIndex({ciudad_destino: 1, estado: 1})
```

- c) Explicar qué sucede internamente en MongoDB cuando se realiza una búsqueda en una colección de gran volumen que no posee índices.

Sin un índice, se realiza un escaneo completo de la colección (lo que internamente se conoce como COLLSCAN), revisando cada documento uno por uno. Esto produce una búsqueda secuencial muy ineficiente, cuyo rendimiento empeora drásticamente a medida que trabajamos con un gran volumen de datos.

2) Implementación práctica:

- a) Crear la base de datos `logistica_db` y la colección `envios`.

```
use logistica_db
```

```
> use logistica_db
< switched to db logistica_db
Atlas atlas-cdbkjq-shard-0 [primary] logistica_db>
```

```
db.createCollection("envios")
```

```
> db.createCollection("envios")
< { ok: 1 }
Atlas atlas-cdbkjq-shard-0 [primary] logistica_db>
```



Pichulman Miguel Angel

- b) Insertar al menos 10 documentos con los campos: numero_seguimiento, estado, ciudad_destino y peso.

```
db.envios.insertMany([
  { numero_seguimiento: 120, estado: "entregado", ciudad_destino: "Mendoza", peso: 25 },
  { numero_seguimiento: 121, estado: "en camino", ciudad_destino: "Córdoba", peso: 15 },
  { numero_seguimiento: 122, estado: "pendiente", ciudad_destino: "San Juan", peso: 5 },
  { numero_seguimiento: 123, estado: "pendiente", ciudad_destino: "San Luis", peso: 28 },
  { numero_seguimiento: 124, estado: "en camino", ciudad_destino: "Cordoba", peso: 13 },
  { numero_seguimiento: 125, estado: "entregado", ciudad_destino: "Tucuman", peso: 10 },
  { numero_seguimiento: 126, estado: "en camino", ciudad_destino: "Salta", peso: 7 },
  { numero_seguimiento: 127, estado: "pendiente", ciudad_destino: "Buenos Aires", peso:
30 },
  { numero_seguimiento: 128, estado: "entregado", ciudad_destino: "Chubut", peso: 35 },
  { numero_seguimiento: 129, estado: "pendiente", ciudad_destino: "Neuquen", peso: 15 }
])
```

```
> db.envios.insertMany([
  { numero_seguimiento: 120, estado: "entregado", ciudad_destino: "Mendoza", peso: 25 },
  { numero_seguimiento: 121, estado: "en camino", ciudad_destino: "Córdoba", peso: 15 },
  { numero_seguimiento: 122, estado: "pendiente", ciudad_destino: "San Juan", peso: 5 },
  { numero_seguimiento: 123, estado: "pendiente", ciudad_destino: "San Luis", peso: 28 },
  { numero_seguimiento: 124, estado: "en camino", ciudad_destino: "Cordoba", peso: 13 },
  { numero_seguimiento: 125, estado: "entregado", ciudad_destino: "Tucuman", peso: 10 },
  { numero_seguimiento: 126, estado: "en camino", ciudad_destino: "Salta", peso: 7 },
  { numero_seguimiento: 127, estado: "pendiente", ciudad_destino: "Buenos Aires", peso: 30 },
  { numero_seguimiento: 128, estado: "entregado", ciudad_destino: "Chubut", peso: 35 },
  { numero_seguimiento: 129, estado: "pendiente", ciudad_destino: "Neuquen", peso: 15 }
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a13a2ebb0f8ccd175db2e94'),
    '1': ObjectId('6a13a2ebb0f8ccd175db2e95'),
    '2': ObjectId('6a13a2ebb0f8ccd175db2e96'),
    '3': ObjectId('6a13a2ebb0f8ccd175db2e97'),
    '4': ObjectId('6a13a2ebb0f8ccd175db2e98'),
    '5': ObjectId('6a13a2ebb0f8ccd175db2e99'),
    '6': ObjectId('6a13a2ebb0f8ccd175db2e9a'),
    '7': ObjectId('6a13a2ebb0f8ccd175db2e9b'),
    '8': ObjectId('6a13a2ebb0f8ccd175db2e9c'),
    '9': ObjectId('6a13a2ebb0f8ccd175db2e9d')
  }
}
Atlas atlas-cdbkjq-shard-0 [primary] logistica_db>
```



ACTIVIDAD 2: Diagnóstico y Evaluación de Rendimiento

1) El equipo de desarrollo nota que una consulta específica está demorando demasiado tiempo en ejecutarse.

- a) Mencionar qué comando de MongoDB permite visualizar si una consulta está realizando un escaneo completo de la colección (COLLSCAN) o utilizando un índice (IXSCAN).

Se debe aplicar `.explain("executionStats")` al final de la consulta de búsqueda. Ej:

```
db.collection.find({ campo: valor }).explain("executionStats")
```

- b) ¿qué significan los campos `executionTimeMillis` y `totalDocsExamined`?

`executionTimeMillis`: tiempo total que tardo en ejecutarse la consulta

`totalDocsExamined`: números de documentos que se revisaron

- c) Según la tabla de problemas detectados, si se observa un uso alto de memoria debido a documentos muy grandes, ¿cuál es la solución recomendada?

Simplificar estructura o usar subcolecciones.

2) Implementación práctica:

- a) Ejecutar una consulta sobre la colección `envios` utilizando `explain("executionStats")` para buscar un paquete por su `numero_seguimiento`.

```
db.envios.find({ numero_seguimiento: 123 }).explain("executionStats")
```



Pichulman Miguel Angel

```
> db.envios.find({ numero_seguimiento: 123 }).explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'logistica_db.envios',
    parsedQuery: {
      numero_seguimiento: {
        '$eq': 123
      }
    },
    indexFilterSet: false,
    queryHash: 'D2CD172B',
    planCacheShapeHash: 'D2CD172B',
    planCacheKey: '8786C56C',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: {
          numero_seguimiento: 1
        },
      },
      indexName: 'numero_seguimiento_1',
      isMultiKey: false,
      multiKeyPaths: {
        numero_seguimiento: []
      },
    },
  },
}
```



Pichulman Miguel Angel

```
,,
  isUnique: false,
  isSparse: false,
  isPartial: false,
  indexVersion: 2,
  direction: 'forward',
  indexBounds: {
    numero_seguimiento: [
      '[123, 123]'
    ]
  }
},
rejectedPlans: []
},
executionStats: {
  executionSuccess: true,
  nReturned: 1,
  executionTimeMillis: 1,
  totalKeysExamined: 1,
  totalDocsExamined: 1,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 1,
    executionTimeMillisEstimate: 0,
    works: 2,
    advanced: 1,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
```

```
docsExamined: 1,
alreadyHasObj: 0,
inputStage: {
  stage: 'IXSCAN',
  nReturned: 1,
  executionTimeMillisEstimate: 0,
  works: 2,
  advanced: 1,
  needTime: 0,
  needYield: 0,
  saveState: 0,
  restoreState: 0,
  isEOF: 1,
  keyPattern: {
    numero_seguimiento: 1
  },
  indexName: 'numero_seguimiento_1',
  isMultiKey: false,
  multiKeyPaths: {
    numero_seguimiento: []
  },
  isUnique: false,
  isSparse: false,
  isPartial: false,
  indexVersion: 2,
  direction: 'forward',
  indexBounds: {
    numero_seguimiento: [
      '[123, 123]'
    ]
  },
  keysExamined: 1,
```




Pichulman Miguel Angel

```
    seeks: 1,
    dupsTested: 0,
    dupsDropped: 0
  }
},
queryShapeHash: 'E40960A0D22C4F8398DF5D2E9C97D431807E79CA05B7875100F9777B52E05D45',
command: {
  find: 'envios',
  filter: {
    numero_seguimiento: 123
  },
  '$db': 'logistica_db'
},
serverInfo: {
  host: 'ac-mtydsi6-shard-00-02.yhxuwj.mongodb.net',
  port: 27017,
  version: '8.0.23',
  gitVersion: 'ccf0d81588377542f00eeecf1b1ffbc095b1eeff'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 16793600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 33554432,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1,
'$clusterTime': {
  clusterTime: Timestamp({ t: 1779674100, i: 12 }),
  signature: {
    hash: Binary.createFromBase64('Rgfv53SAdi0Uq5vHgKhEvVKYwu0=', 0),
    keyId: 7582612401418142000
  }
},
operationTime: Timestamp({ t: 1779674100, i: 12 })
}
Atlas atlas-cdbkjq-shard-0 [primary] logistica_db>
```



Pichulman Miguel Angel

- b) Analizar el resultado: Identificar el stage y verificar si se utilizó el índice creado en la Actividad 1.

Al analizar el plan de ejecución, el campo stage muestra IXSCAN, lo que confirma que la base de datos utilizó de manera eficiente el índice simple creado en la Actividad 1 en lugar de escanear toda la colección.

- c) Ejecutar el comando stats() sobre la colección y reportar el tamaño total de los índices (totalIndexSize).

```
sharded: false,
size: 1065,
count: 10,
numOrphanDocs: 0,
storageSize: 36864,
totalIndexSize: 57344,
totalSize: 94208,
indexSizes: { _id_: 36864, numero_seguimiento_1: 20480 },
avgObjSize: 106,
ns: 'logistica_db.envios',
nindexes: 2,
scaleFactor: 1
}
Atlas atlas-cdbkjq-shard-0 [primary] logistica_db>
```

El tamaño informado es 57344.

ACTIVIDAD 3: Procesamiento Avanzado con Agregaciones

1) Se requiere generar reportes financieros y operativos de los envíos realizados.

- a) ¿Cuál es la función del Pipeline de Agregación y qué significa que funcione como una "tubería"?

La función del Pipeline de Agregación es procesar, transformar y resumir grandes volúmenes de datos. Se le llama 'tubería' porque está compuesto por etapas secuenciales (stages), donde la salida de una etapa se convierte en la entrada de la siguiente, y cada etapa aplica una operación específica sobre los documentos.

- b) Describir brevemente para qué sirven las etapas \$match, \$group y \$project.

\$match: filtra documentos (WHERE en SQL)

\$group: Agrupa documentos por un campo y aplica funciones de agregado.

\$project: selecciona o transforma campos (SELECT en SQL)

- c) ¿Qué operador de agregación utilizarías para unir la colección envios con una colección de clientes?

Para unir la colección de envíos con una colección de clientes utilizaría el operador \$lookup, el cual permite realizar un 'join' integrando información de múltiples colecciones en un solo documento de salida.



Pichulman Miguel Angel

2) Implementación práctica:

- a) Realizar una agregación que filtre (\$match) los envíos con estado "Entregado" y los agrupe (\$group) por ciudad_destino para contar cuántos se entregaron en cada ciudad.

```
db.envios.aggregate([
  { $match: { estado : "entregado" } },
  { $group: { _id: "$ciudad_destino", total: { $sum: 1 } } }
])
```

```
> db.envios.aggregate([
  { $match: { estado : "entregado" } },
  { $group: { _id: "$ciudad_destino", total: { $sum: 1 } } }
])
< {
  _id: 'Mendoza',
  total: 1
}
{
  _id: 'Chubut',
  total: 1
}
{
  _id: 'Tucuman',
  total: 1
}
Atlas atlas-cdbkjq-shard-0 [primary] logistica_db>
```

- b) Utilizar una función matemática para calcular el peso total de todos los envíos en la colección.

```
db.envios.aggregate([
  { $group: { _id: null, peso_total: { $sum: "$peso" } } }
])
```

```
> db.envios.aggregate([
  { $group: { _id: null, peso_total: { $sum: "$peso" } } }
])
< {
  _id: null,
  peso_total: 183
}
Atlas atlas-cdbkjq-shard-0 [primary] logistica_db>
```




Pichulman Miguel Angel

• c) Crear un pipeline que utilice \$project para mostrar el numero_seguimiento en mayúsculas (usando \$toUpper) y el estado.

```
db.envios.aggregate([
  {
    $project: {
      _id: 0,
      estado: 1,
      numero_seguimiento: { $toUpper: "$numero_seguimiento" }
    }
  }
])
```

```
> db.envios.aggregate([
  {
    $project: {
      _id: 0,
      estado: 1,
      numero_seguimiento: { $toUpper: "$numero_seguimiento" }
    }
  }
])
< {
  estado: 'entregado',
  numero_seguimiento: '120'
}
{
  estado: 'en camino',
  numero_seguimiento: '121'
}
{
  estado: 'pendiente',
  numero_seguimiento: '122'
}
{
  estado: 'pendiente',
  numero_seguimiento: '123'
}
{
  estado: 'en camino',
  numero_seguimiento: '124'
}
{
  estado: 'entregado',
  numero_seguimiento: '125'
}
{
  estado: 'en camino',
  numero_seguimiento: '126'
```



Pichulman Miguel Angel

```
}
{
  estado: 'pendiente',
  numero_seguimiento: '127'
}
{
  estado: 'entregado',
  numero_seguimiento: '128'
}
{
  estado: 'pendiente',
  numero_seguimiento: '129'
}
Atlas atlas-cdbkjq-shard-0 [primary] logistica_db>
```

- d) Implementar una etapa de \$addFields o \$project que use un condicional \$cond para crear un campo nuevo llamado "tipo_carga": si el peso es mayor a 20, "Pesada", de lo contrario "Ligera".

```
db.envios.aggregate([
  {
    $addFields: {
      tipo_carga: {
        $cond: {
          if: { $gt: ["$peso", 20] },
          then: "pesada",
          else: "ligera"
        }
      }
    }
  }
])
```



Pichulman Miguel Angel

```
> db.envios.aggregate([
  {
    $addFields: {
      tipo_carga: {
        $cond: {
          if: { $gt: ["$peso", 20] },
          then: "pesada",
          else: "ligera"
        }
      }
    }
  }
])
< {
  _id: ObjectId('6a13a2ebb0f8ccd175db2e94'),
  numero_seguimiento: 120,
  estado: 'entregado',
  ciudad_destino: 'Mendoza',
  peso: 25,
  tipo_carga: 'pesada'
}
{
  _id: ObjectId('6a13a2ebb0f8ccd175db2e95'),
  numero_seguimiento: 121,
  estado: 'en camino',
  ciudad_destino: 'Córdoba',
  peso: 15,
  tipo_carga: 'ligera'
}
{
  _id: ObjectId('6a13a2ebb0f8ccd175db2e96'),
  numero_seguimiento: 122,
  estado: 'pendiente',
  ciudad_destino: 'San Juan',
  peso: 5,
  tipo_carga: 'ligera'
}
{
  _id: ObjectId('6a13a2ebb0f8ccd175db2e97'),
  numero_seguimiento: 123,
  estado: 'pendiente',
  ciudad_destino: 'San Luis',
  peso: 28,
  tipo_carga: 'pesada'
}
```

```
}
{
  _id: ObjectId('6a13a2ebb0f8ccd175db2e98'),
  numero_seguimiento: 124,
  estado: 'en camino',
  ciudad_destino: 'Cordoba',
  peso: 13,
  tipo_carga: 'ligera'
}
{
  _id: ObjectId('6a13a2ebb0f8ccd175db2e99'),
  numero_seguimiento: 125,
  estado: 'entregado',
  ciudad_destino: 'Tucuman',
  peso: 10,
  tipo_carga: 'ligera'
}
{
  _id: ObjectId('6a13a2ebb0f8ccd175db2e9a'),
  numero_seguimiento: 126,
  estado: 'en camino',
  ciudad_destino: 'Salta',
  peso: 7,
  tipo_carga: 'ligera'
}
{
  _id: ObjectId('6a13a2ebb0f8ccd175db2e9b'),
  numero_seguimiento: 127,
  estado: 'pendiente',
  ciudad_destino: 'Buenos Aires',
  peso: 30,
  tipo_carga: 'pesada'
}
{
  _id: ObjectId('6a13a2ebb0f8ccd175db2e9c'),
  numero_seguimiento: 128,
  estado: 'entregado',
  ciudad_destino: 'Chubut',
  peso: 35,
  tipo_carga: 'pesada'
}
{
  _id: ObjectId('6a13a2ebb0f8ccd175db2e9d'),
  numero_seguimiento: 129,
  estado: 'pendiente',
  ciudad_destino: 'Neuquen',
  peso: 15,
  tipo_carga: 'ligera'
}
}
```

Atlas atlas-cdbkjq-shard-0 [primary] logistica_db>